

Codebook-Based Acceleration for Ternary Language Model Inference

Tomas Sousa Pereira
tsousape@nd.edu
University of Notre Dame
South Bend, Indiana, USA

Ruibao Wang
wangrb18@gmail.com
University of Notre Dame
South Bend, Indiana, USA

Ikenna Nwozo
inwozo@nd.edu
University of Notre Dame
South Bend, Indiana, USA

Haoran Geng
hgeng@nd.edu
University of Notre Dame
South Bend, Indiana, USA

X. Sharon Hu
shu@nd.edu
University of Notre Dame
South Bend, Indiana, USA

Abstract

Emerging 1.58-bit large language models such as BitNet restrict weights to $\{-1, 0, +1\}$, dramatically reducing model size. Yet commodity hardware fails to fully exploit this structure: each ternary value carries only $\log_2(3) \approx 1.58$ bits of information but must occupy a 2-bit container, wasting 21% of memory bandwidth. Furthermore the lack of native mixed-precision support forces costly runtime dequantization. We present a hardware-software co-design that addresses both inefficiencies. On the software side, we compress ternary weights to 1.5 bits per weight via a 64-entry codebook with 6-bit indices, sacrificing only 5.4% of representational capacity; Vector Quantization aware fine-tuning retains 92.6% of baseline accuracy across six benchmarks. On the hardware side, we design TERINT-GEMV, a multiplier-free bit-serial accelerator that ingests the 6-bit indices directly through an on-chip lookup table, computing ternary $\times$ INT8 dot products with dynamic zero-skipping. Synthesized using a 28 nm commercial PDK, TERINT-GEMV achieves 99.44 TOPS/W, outperforming recently published BitNet accelerators by up to 7.2 $\times$ in energy efficiency, and delivers 0.065 J/token with a 2.1 $\times$ throughput gain over the leading CPU-based ternary runtime at one fifth the power.

CCS Concepts

• Computer systems organization $\rightarrow$ Neural networks; • Hardware $\rightarrow$ Arithmetic and datapath circuits.

Keywords

Low-bit LLM, Software-hardware co-design, Accelerator

ACM Reference Format:

Tomas Sousa Pereira, Ruibao Wang, Ikenna Nwozo, Haoran Geng, and X. Sharon Hu. 2026. Codebook-Based Acceleration for Ternary Language Model Inference. In *ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED '26)*, August 05–07, 2026, Evanston, IL, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3816440.3818578>

1 Introduction

Large Language Models (LLMs) have become central to modern AI applications, but their deployment faces a critical energy challenge. Inference dominates the energy footprint, accounting for over 90% of lifecycle power consumption [14]. The problem is memory-bound: a single DRAM access consumes 1000 $\times$ to 2000 $\times$ more energy than an arithmetic operation [4], so most of the power budget goes to moving weights between memory and compute units.

Model quantization reduces this energy burden by shrinking the data that must move from memory. While post-training quantization to INT8 or INT4 offers moderate gains, more aggressive schemes push further. Binary networks [5] replace multiplications with XNOR operations but suffer significant accuracy loss on large models [23]. Ternary weight networks [8] recover much of that accuracy by introducing a third state, $\{-1, 0, +1\}$, which allows weights to be explicitly zeroed, producing naturally sparse matrices while still eliminating the need for full multiplications.

BitNet [10] demonstrates that ternary constraints can scale to billion-parameter LLMs when models are trained from scratch with weights restricted to $\{-1, 0, +1\}$ and activations kept at INT8 precision, and recent work confirms that ternary models obey the same scaling laws as their full-precision counterparts [19]. The resulting models are roughly 10 $\times$ smaller than FP16, directly cutting DRAM data movement energy, and their 40–60% natural zero-weight sparsity opens the door to skipping large fractions of computation.

However, commodity hardware cannot exploit these benefits due to two limitations. First, CPUs/GPUs lack native support for mixed-precision operations (ternary weights $\times$ INT8 activations), forcing costly runtime dequantization that erodes compute efficiency [22]. Second, though a ternary value carries only $\log_2(3) \approx 1.58$ bits of information, existing approaches store each weight in a 2-bit container, wasting 0.42 bits per weight, which leads to substantial storage and energy overhead across billions of parameters.

To address these inefficiencies, we propose a software-hardware co-design that eliminates both the precision alignment overhead and the bit-level storage waste. On the software side, we introduce a codebook-based compression scheme that reduces ternary weights from 2 bits to 1.5 bits per weight by intelligently grouping them and exploiting their information-theoretic structure. On the hardware side, we design TERINT-GEMV, a custom bit-serial accelerator that natively executes ternary $\times$ INT8 operations without dequantization, while also leveraging sparsity to gate unnecessary



This work is licensed under a Creative Commons Attribution 4.0 International License. *ISLPED '26, Evanston, IL, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2748-1/2026/08
<https://doi.org/10.1145/3816440.3818578>

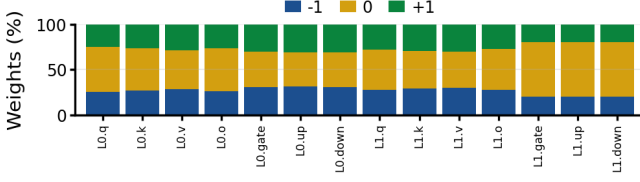


Figure 1: Ternary weight distribution across the first two layers of a trained BitNet model.

computation dynamically. Together, these innovations enable significant power savings over existing BitNet implementations.

Our main contributions include (1) a mathematical formulation of the bit-waste problem in ternary models and an information-theoretic framework for minimizing storage overhead; (2) a uniqueness-driven codebook pruning algorithm that compresses ternary weights to 1.5 bits per weight with minimal accuracy loss; (3) TERINT-GEMV, a bit-serial hardware accelerator with native ternary $\times$ INT8 matrix-vector and matrix-matrix multiplication and dynamic zero-skipping for power efficiency.

We evaluate our system using the lm-evaluation-harness framework [3] on standard LLM benchmarks including ARC-Easy, ARC-Challenge, CommonsenseQA, MMLU, and GSM8K. We assess hardware performance through RTL synthesis using a 28 nm commercial PDK. Our results demonstrate a 90% reduction in energy per token (including DRAM access energy) and a 2.1 $\times$ throughput improvement over T-MAC, the leading CPU-based BitNet runtime.

2 Motivation and Related Work

BitNet [10] and similar 1.58-bit architectures constrain each weight to $w \in \{-1, 0, +1\}$ while keeping activations at INT8 precision. As shown in Figure 1, trained BitNet models exhibit 40–60% zero weights, creating natural structural sparsity that hardware can exploit to skip computation and reduce dynamic power.

Two inefficiencies prevent commodity hardware from exploiting ternary models. First, each ternary value carries only $\log_2(3) \approx 1.585$ bits of information but must be stored in a 2-bit container, wasting 0.415 bits per weight; for a 7B-parameter model this accumulates to ~ 365 MB of unnecessary memory traffic per inference step. Second, CPUs and GPUs lack native mixed-precision ternary $\times$ INT8 arithmetic, forcing *runtime dequantization* that erodes bandwidth and compute advantages [22].

We review prior work addressing these inefficiencies from two directions: software runtimes that adapt CPUs and GPUs for ternary inference, and dedicated accelerators for highly-quantized models.

Software Approaches for Ternary Models. Compressing weights reduces DRAM traffic and, since DRAM energy scales linearly with bits transferred, directly lowers inference power. Software frameworks such as T-MAC [22] and BitNet.cpp [21] run 1.58-bit models on commodity CPUs, with T-MAC replacing MAC operations with Look Up Table (LUT) reads and LUT Tensor Core [12] extending this to a software-hardware co-design with a bit-serial LUT datapath. However, all these approaches still use 2-bit storage, leaving the 0.415-bit waste unaddressed.

On the algorithmic side, vector quantization methods like AQLM [2] group weights into codebooks, achieving sub-2-bit compression. However, their decoding requires floating-point

operations to reconstruct weights before computation, creating an on-chip bottleneck that counteracts the energy saved from reduced data movement. The key challenge is making decoding cheap enough that it does not negate the compression benefit.

Accelerators for Mixed-Precision and Ternary Inference. Hardware acceleration for neural network inference can be organized by the precision of the two operands involved in multiplication: (1) both operands at higher precision (FP or INT), (2) both operands at binary or ternary precision (symmetric), and (3) the asymmetric case where ternary weights are paired with higher-precision INT activations. Our work falls in the third category.

In the first category, bit-serial architectures such as BitFusion [18], Loom [9], and BISDU [11] define precision by cycle count rather than fixed datapath width, allowing the same hardware to handle operands from 1-bit to 8-bit without under-utilization. Halving the precision roughly doubles throughput, a linear scaling that bit-parallel multipliers (whose area grows quadratically with width) cannot match, and these designs have demonstrated significant energy and area gains over fixed-precision MAC arrays. However, because they are designed for general-purpose precision flexibility, they treat ternary weights as just another low-precision operand and do not exploit properties specific to ternary LLMs.

In the second category, accelerators such as CUTIE [16] target networks where both weights and activations are quantized to binary or ternary precision, achieving 3.1 PetaOp/s/W with better-than-binary energy efficiency [13]. However, BitNet operates differently: weights are ternary but activations remain at INT8, creating an asymmetric mixed-precision workload that these symmetric designs cannot handle natively.

The third category, which matches the BitNet workload, can be further divided into two sub-approaches: extending commodity CPUs with lightweight ISA support, and building dedicated accelerators. On the CPU side, BNRV [6] adds a SIMD extension to RISC-V for BitNet inference, and xTern [15] targets energy-efficient ternary execution on RISC-V edge systems. These approaches preserve the familiar programming model but are ultimately limited by the processor’s fixed datapath width and general-purpose control overhead. On the accelerator side, Platinum [17] introduces a lightweight ASIC that utilizes LUTs for BitNet b1.58 inference. Slim-Llama [7] demonstrates a milliwatt-scale full-digital ASIC that leverages a sparsity-aware lookup table to dynamically skip zero-weight operations. BITROM [24] introduces a Compute-in-Read-Only-Memory architecture that stores two ternary weights per transistor on-chip, eliminating the energy overhead of runtime weight reloading from external DRAM. Despite their architectural innovations, all accelerators in this category share a common limitation: they operate under the assumption of standard 2-bit weight storage. By focusing primarily on the compute and immediate caching layers, they leave the global memory bandwidth overhead, driven by the 0.415-bit storage waste, entirely unaddressed.

3 Our Approach

3.1 Representation Compression

Our objective is to close the gap between the theoretical information content of ternary weights and their physical storage cost. A codebook-based approach is natural for this problem: since each

Table 1: Analysis of group sizes for ternary codebook compression. Information loss ΔH is per weight.

$ g $	$3^{ g }$	K	Bits	Pruned	Bits/Wt	ΔH	$\delta_{\max}$
3	27	16	4	11 (41%)	1.33	0.252	12
4	81	64	6	17 (21%)	1.50	0.085	16
5	243	128	7	115 (47%)	1.40	0.185	20
6	729	512	9	217 (30%)	1.50	0.085	24

ternary weight carries only $\log_2(3) \approx 1.585$ bits of information yet must occupy a 2-bit container on commodity hardware, we can recover the wasted capacity by mapping groups of weights to compact integer indices via a shared codebook. Grouping is key because the fractional bit waste per weight accumulates over a group, and for certain group sizes the total information content aligns more closely with a power-of-two codebook, making nearly lossless compression possible.

The central challenge is designing the right codebook. We formalize this as a constrained subset selection problem. Let $\mathbf{W} \in \{-1, 0, +1\}^{m \times n}$ be a ternary weight matrix partitioned into non-overlapping groups $g_i \in \{-1, 0, +1\}^{|g|}$, where $|g|$ denotes the group size and there are $G = mn/|g|$ groups in total. The full pattern set $\mathcal{P}_{|g|} = \{-1, 0, +1\}^{|g|}$ has $3^{|g|}$ entries, but hardware-efficient encoding requires a codebook $C \subseteq \mathcal{P}_{|g|}$ of size $K = 2^b$. When $K < 3^{|g|}$, some patterns must be pruned, and each pruned pattern is replaced by its nearest retained code, yielding an approximation $\tilde{\mathbf{W}} = \mathbf{W} + \mathbf{E}$ where $\mathbf{E}$ is the perturbation matrix. The output error of a linear layer $\mathbf{y} = \mathbf{W}\mathbf{x}$ is bounded by:

$$\|\mathbf{E}\mathbf{x}\|_2 \leq \|\mathbf{E}\|_F \cdot \|\mathbf{x}\|_2, \quad \text{with } \|\mathbf{E}\|_F^2 = G \cdot D(C) \quad (1)$$

where $D(C)$ is the average codebook distortion. Minimizing this distortion is the core optimization target.

Solving this problem involves three design decisions, and will be discussed individually in more detail below.

- (1) **Group size selection** (§3.1.1): choosing $|g|$ to balance compression ratio, pruning severity, and hardware complexity;
- (2) **Codebook design** (§3.1.2): deciding which $3^{|g|} - K$ patterns to prune to minimize distortion;
- (3) **Code assignment policy** (§3.1.3): mapping each pruned pattern to its replacement code.

Finally, we apply **codebook-aware fine-tuning** (§3.1.4) to jointly optimize the model weights and codebook assignments, recovering accuracy lost during compression.

3.1.1 Group size selection. The group size $|g|$ controls the trade-off between information loss and codebook size, which determines how many patterns must be pruned. It also sets the number of weights the hardware must process per index, which drives LUT area and datapath width. For each candidate $|g|$, the natural codebook size is $K = 2^{\lfloor \log_2(3^{|g|}) \rfloor}$, requiring $3^{|g|} - K$ patterns to be pruned, and the effective encoding rate is $\lfloor \log_2(3^{|g|}) \rfloor / |g|$ bits per weight. Table 1 summarizes this analysis.

The information loss per weight for group size $|g|$ is:

$$\Delta H(|g|) = \frac{H_{\max}(|g|) - \log_2(K)}{|g|} = \frac{|g| \log_2(3) - \lfloor |g| \log_2(3) \rfloor}{|g|} \quad (2)$$

Applying Equation (2), for $|g| = 3$ the loss is high (0.252 bits/weight, or 15.9%) and 41% of patterns are discarded. For $|g| = 5$, although the bits-per-weight is lower (1.40), nearly half the patterns must be pruned, sharply increasing quantization distortion. Both $|g| = 4$ and $|g| = 6$ achieve 1.50 bits/weight with only 0.085 bits/weight information loss (5.4%). However, $|g| = 6$ requires a 512-entry codebook and 9-bit indices, increasing on-chip LUT area and decode complexity. We select $|g| = 4$ as the design point: it prunes the fewest fraction of patterns at its compression level, uses compact 6-bit indices, and maps cleanly onto byte-aligned memory accesses (four indices per three bytes).

In practice, the actual loss is even smaller because trained BitNet models exhibit highly non-uniform weight distributions, where many of the 81 patterns are rare. The remaining challenge is selecting which 17 patterns to prune and how to reassign the affected weight groups.

3.1.2 Analytically Motivated Codebook Selection. Given $|g| = 4$, we must reduce the pattern vocabulary from 81 to 64, discarding 17 patterns. The choice of *which* patterns to prune directly controls the quantization distortion $D(C)$. We formalize this as a subset selection problem and evaluate three strategies.

For any pruned pattern $\mathbf{p} \in \mathcal{P}_4 \setminus C$, its quantization error is the squared distance to the nearest retained code, $\delta(\mathbf{p}) = \min_{\mathbf{c} \in C} \|\mathbf{p} - \mathbf{c}\|_2^2$. Since all entries lie in $\{-1, 0, +1\}$, the per-element difference is at most 2, giving a worst-case bound of $\delta(\mathbf{p}) \leq 4 \cdot 2^2 = 16$. The total expected distortion is $D(C) = \sum_{\mathbf{p}} p_{\mathbf{p}} \delta(\mathbf{p})$, summed over pruned patterns $\mathbf{p} \in \mathcal{P}_4 \setminus C$, where $p_{\mathbf{p}}$ is the pattern frequency. A good codebook minimizes $D(C)$ by ensuring that pruned patterns are both rare ($p_{\mathbf{p}}$ small) and close to retained codes ($\delta(\mathbf{p})$ small).

We explore three selection strategies:

- **Uniqueness (Most Distant (MD)):** Retains the K patterns that maximize the minimum pairwise Euclidean distance within the codebook: $C^* = \arg \max_{C \subseteq \mathcal{P}_{|g|}, |C|=K} \min_{\mathbf{c}_i, \mathbf{c}_j \in C, i \neq j} \|\mathbf{c}_i - \mathbf{c}_j\|_2^2$. This ensures the retained codes span the widest possible region of the weight space, minimizing the $\delta(\mathbf{p})$ terms by guaranteeing every pruned pattern has a geometrically close neighbor in the codebook.
- **Redundancy (Most Redundant (MR)):** Iteratively removes the pattern whose minimum distance to the remaining codebook is smallest: at each step, remove $\arg \min_{\mathbf{p} \in C} \min_{\mathbf{c} \in C \setminus \{\mathbf{p}\}} \|\mathbf{p} - \mathbf{c}\|_2^2$, until $|C| = K$. This greedy approach targets geometric redundancy, removing the most “replaceable” patterns first.
- **Frequency (Least Common (LC)):** Ranks all patterns by their empirical frequency $p_{\mathbf{p}}$ in the trained model and retains the K most frequent: $C^* = \arg \max_{C \subseteq \mathcal{P}_{|g|}, |C|=K} \sum_{\mathbf{c} \in C} p_{\mathbf{c}}$. This directly minimizes the $p_{\mathbf{p}}$ terms in the distortion sum.

Each strategy targets a different term in the distortion sum: Uniqueness and Redundancy minimize the quantization error $\delta(\mathbf{p})$ by ensuring good spatial coverage, but may retain rare patterns at the expense of frequent ones. Frequency minimizes the probability mass $p_{\mathbf{p}}$, but frequent patterns often cluster near zero, wasting codebook capacity on redundant entries. We evaluate all three empirically in §4.1.

In addition to the selection strategy, we investigate two codebook granularities: a single *global codebook* shared across all layers versus *layer-wise codebooks* tailored to each layer’s weight distribution. The global approach minimizes hardware complexity (one LUT per accelerator), while layer-wise codebooks can achieve tighter compression by adapting to per-layer statistics at the cost of increased on-chip storage.

3.1.3 Pattern Replacement Strategies. Once the 64-pattern codebook C is fixed, every weight group whose pattern falls outside C must be mapped to a valid code. Given codebook C , each weight group is assigned $\hat{g}_i = \arg \min_{c \in C} d(g_i, c)$, where $d(\cdot, \cdot)$ is a distance function. We evaluate three strategies:

- **Zero-Snap:** Replaces all invalid patterns with $[0, 0, 0, 0]$, aggressively increasing sparsity. This benefits hardware efficiency through zero-skipping but risks distorting the learned weight distribution. a
- **Closest-L2:** Uses $d(g, c) = \|g - c\|_2^2$ to assign each pruned group to its nearest Euclidean neighbor. This minimizes the Frobenius norm of the perturbation $\|E\|_F$ and provides the tightest bound on layer output error (Equation 1).
- **Closest-Hamming:** Uses $d(g, c) = \sum_{k=1}^{|g|} 1[g_k \neq c_k]$, assigning to the pattern with the fewest element-wise changes. This is computationally cheaper and preserves local weight structure, though it ignores the magnitude of each change.

We evaluate all three replacement strategies experimentally in combination with each codebook selection method in §4.1.

3.1.4 Codebook-Aware Fine-Tuning. After selecting a codebook (§3.1.2) and applying a replacement strategy (§3.1.3), the compressed model has not yet been trained under the codebook constraint. To recover the accuracy lost from pattern pruning and replacement, we fine-tune the model using an objective inspired by VQ-VAE [20], adapted from the latent-space setting to operate directly on the weight domain.

The model is optimized under a combined loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{LM}}(\theta) + \lambda \mathcal{L}_{\text{VQ}}(\theta, C)$$

where $\mathcal{L}_{\text{LM}}$ is the standard language modeling loss and the $\mathcal{L}_{\text{VQ}}$ loss consists of two terms:

$$\mathcal{L}_{\text{VQ}} = \underbrace{\frac{1}{G} \sum_i \|\text{sg}[g_i] - \hat{g}_i\|_2^2}_{\text{codebook loss}} + \beta \underbrace{\frac{1}{G} \sum_i \|g_i - \text{sg}[\hat{g}_i]\|_2^2}_{\text{commitment loss}}$$

Here $\text{sg}[\cdot]$ is the stop-gradient operator. We include both terms in $\mathcal{L}_{\text{VQ}}$ because the optimization must move both sides toward each other: the codebook loss pulls codebook entries toward the current weight clusters, while the commitment loss pushes weights toward their assigned codes. The hyperparameter β controls the relative strength of these two forces.

Since the quantization step $g_i \rightarrow \hat{g}_i$ is non-differentiable, gradients are propagated through it using the Straight-Through Estimator (STE) [1]:

$$\frac{\partial \mathcal{L}}{\partial g_i} \approx \frac{\partial \mathcal{L}}{\partial \hat{g}_i}$$

The STE provides a biased but low-variance gradient estimate, and the bias vanishes as the codebook distortion $D(C) \rightarrow 0$. Training

Algorithm 1 BSFU for one weight group.

```

1: Input: 6-bit index  $i$ , INT8 activations  $\mathbf{x} = [x_0, x_1, x_2, x_3]$ 
2:  $(\text{nz\_mask}, \text{sgn\_mask}) \leftarrow \text{CodebookLUT}[i]$  ▷ Decode
3:  $\text{acc} \leftarrow 0$ 
4: for  $t = 0$  to 7 do ▷ Bit-serial loop
5:    $\mathbf{x}_t \leftarrow \text{bit } t \text{ of } [x_0, x_1, x_2, x_3]$  ▷ 4-bit slice
6:    $\text{active} \leftarrow \mathbf{x}_t \wedge \text{nz\_mask}$  ▷ S1: Masking
7:    $\text{pos} \leftarrow \text{active} \wedge \text{sgn\_mask}$  ▷ S2: Sign sep.
8:    $\text{neg} \leftarrow \text{active} \wedge \neg \text{sgn\_mask}$ 
9:    $\text{acc} \leftarrow \text{acc} + (\text{popcount}(\text{pos}) - \text{popcount}(\text{neg})) \ll t$  ▷ S3: Accum.
10: end for
11: return  $\text{acc}$ 

```

alternates between reassigning each weight group to its nearest codebook entry and performing gradient descent on $\mathcal{L}_{\text{total}}$, progressively tightening the error bound in Equation 1.

3.2 Hardware accelerator design

To fully exploit the 6-bit codebook representation produced by our compression pipeline, we design TERINT-GEMV, an accelerator that ingests these codebook indices directly and performs ternary $\times$ INT8 matrix-vector and matrix-matrix multiplication without any multiplier hardware.

The key observation enabling this design is that ternary weights eliminate the need for multiplication entirely: since $w \in \{-1, 0, +1\}$, the product $w \cdot x$ reduces to adding x , subtracting x , or skipping. We exploit this by designing a *Bit-Serial Functional Unit* (BSFU) that processes activations one bit at a time using only logic gates and adders. Below, we first detail the BSFU operation, then describe how the Codebook LUT translates indices into control signals, and finally show how these components are composed into 2D array.

3.2.1 Bit-Serial Functional Unit (BSFU). The BSFU computes the dot product $\mathbf{w} \cdot \mathbf{x} = \sum_{k=0}^3 w_k \cdot x_k$ for one weight group without any multiplier. For a ternary weight w and a B -bit integer activation $x = \sum_{t=0}^{B-1} x_t \cdot 2^t$, the product decomposes as:

$$\mathbf{w} \cdot \mathbf{x} = \sum_{t=0}^{B-1} (w \cdot x_t) \cdot 2^t$$

Since $x_t \in \{0, 1\}$ and $w \in \{-1, 0, +1\}$, each term $w \cdot x_t$ reduces to adding, subtracting, or skipping a single-bit value. The BSFU processes INT8 activations one bit at a time over $B = 8$ cycles, performing three steps per cycle (Algorithm 1); Figure 2 shows the corresponding hardware block diagram.

Algorithm 1 details the steps for a weight group of size 4. Using the weight group $\mathbf{w} = [+1, -1, 0, +1]$ as an example: the Codebook LUT (§3.2.2) encodes it as $\text{nz_mask} = 1101_2$ and $\text{sgn_mask} = 1001_2$. Each cycle extracts one activation bit, masks zero-weight positions (S1), separates positive and negative contributions (S2), and accumulates the popcount difference shifted by bit position (S3). After 8 cycles, the accumulator holds the exact dot product.

3.2.2 Pattern Codebook LUT and Zero-Skipping. As described in §3.2.1, the BSFU operates on two 4-bit control masks: the nz_mask and the sgn_mask . These masks are $|g| = 4$ bits wide because each encodes one bit of information per weight position in the group.

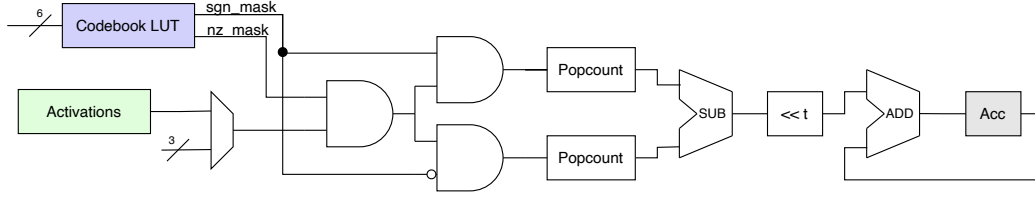


Figure 2: BSFU datapath. The Codebook LUT decodes the 6-bit weight index into `sgn_mask` and `nz_mask`; each activation bit-slice is gated through two AND–Popcount paths, subtracted, barrel-shifted by t , and accumulated into the 32-bit Acc over $B = 8$ cycles.

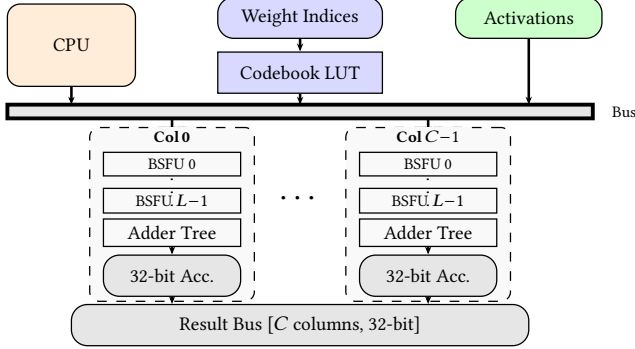


Figure 3: Top-level TERINT-GEMV architecture. Codebook masks and activations are broadcast to $C \times L$ BSFUs, with each column feeding an adder tree and accumulator.

The Codebook LUT outputs these masks by translating each 6-bit index into the corresponding pair. Storing the masks directly (rather than the ternary values $\{-1, 0, +1\}^4$, which would require 8 bits) exploits the fact that two bits per weight suffice to capture the three states: non-zero or zero, and if non-zero, positive or negative.

Continuing the example from §3.2.1, the pattern $[+1, -1, 0, +1]$ is encoded as `nz_mask` = 1101₂ and `sgn_mask` = 1001₂. The codebook also encodes sparsity: the all-zero pattern $[0, 0, 0, 0]$ produces `nz_mask` = 0000₂, which gates every activation bit, so no switching occurs in the downstream logic. More broadly, for any pattern with z zero positions out of 4, the AND masking in the BSFU prevents $z/4$ of the datapath from toggling, reducing dynamic power proportionally. Since BitNet models exhibit 40–60% zero weights (Figure 1), this implicit zero-skipping provides substantial dynamic power savings without dedicated sparsity detection hardware.

3.2.3 Processing Element and Array Organization. A single Codebook LUT (§3.2.2) is shared across the array, decoding each 6-bit weight index into dual-mask control signals. Each BSFU (§3.2.1) consumes these masks with the activation data to produce a dot product (Figure 2). Multiple BSFUs are organized into a 2D array (Figure 3), where each column of L BSFUs shares an activation bus and processes weight groups in parallel. A reduction tree within each column sums the BSFU outputs into a 32-bit accumulator.

4 Experimental Evaluation

Experimental Setup. All experiments use the `bitnet-b1.58-2B-4T` model released by Microsoft, evaluated with the LM Evaluation Harness [3]. Throughout, weights are ternary $\{-1, 0, +1\}$ and

activations are quantized to INT8, following the BitNet b1.58 specification; our codebook compression applies only to the weights and leaves the activation precision unchanged. For codebook-aware fine-tuning, we construct a training dataset from held-out partitions of ARC, WinoGrande, CommonsenseQA, MMLU, and GSM8K, ensuring no overlap with evaluation splits [3]. Software and RTL code are open-sourced at <https://github.com/Notre-Dame-HW-SW-Codesign-Lab/TERINT-GEMV>. We evaluate three aspects: the impact of codebook compression on accuracy (§4.1), the area, energy efficiency and throughput of the BSFU-based accelerator (§4.2), and end-to-end benchmarking against recently published work on accelerating BitNet (§4.3).

4.1 Accuracy Evaluation

We evaluate how codebook compression affects accuracy by measuring benchmark performance across all combinations of codebook selection strategies (§3.1.2) and replacement policies (§3.1.3), and for both global and per-layer codebooks. Table 2 reports results.

Three findings stand out. LC pruning with L2 or Hamming replacement consistently outperforms MD and MR before fine-tuning. Because ternary weights concentrate around zero, the most frequent patterns carry the bulk of the model’s representational load; discarding them for geometrically diverse but rare patterns destroys more information than it preserves. VQ fine-tuning recovers most of the remaining loss, bringing ARC-Easy from 71.9% to 73.6% against a 74.8% baseline, though reasoning-heavy tasks like CommonsenseQA remain more sensitive. Recovery works because the joint LM + VQ objective lets the weights gradually shift toward the 64 retained codes, learning to “work around” the constraint rather than being forced into it at once. Neither granularity dominates: taking the best configuration per benchmark, per-layer codebooks lead on four of seven tasks (WinoGrande, CommonsenseQA, MMLU, GSM8K) and the global codebook on the other three (ARC-E, ARC-C, OBQA), typically within one to two points. Per-layer codebooks adapt to each layer’s distribution, but the global codebook stays competitive while needing only a single on-chip LUT shared across the array (§3.2), which we adopt in our design.

Overall, the best fine-tuned configuration retains on average 92.6% of the baseline accuracy across six benchmarks, with an average loss of 4.5 percentage points. GSM8K is the outlier, dropping from 58.3% to 32.2% (a 26.1pp loss), because its multi-step reasoning chains amplify small weight perturbations into cascading errors.

Table 2: Accuracy after codebook compression for global (G) and per-layer (L) codebooks; bold marks the best per benchmark. LC/MR/MD = Least Common/Most Redundant/Most Distant; 0/L2/HD = Zero-Snap/Closest-L2/Closest-Hamming; FT = VQ fine-tuning.

		Base	LC			MR			MD			FT
			0	L2	HD	0	L2	HD	0	L2	HD	
ARC-E	G	74.8	29.8	71.9	71.9	24.5	25.3	34.7	25.0	40.0	34.7	73.6
	L		29.9	71.9	71.9	49.4	67.1	55.4	29.0	67.9	58.3	72.2
ARC-C	G	49.9	24.2	46.5	46.5	25.6	25.2	31.0	24.6	29.5	31.0	45.7
	L		24.4	44.9	44.9	28.3	42.8	32.2	22.8	43.5	32.2	42.9
OBQA	G	41.6	28.4	40.2	40.2	27.6	30.6	31.6	28.8	33.0	31.6	40.0
	L		28.0	39.9	39.0	26.6	37.2	31.2	26.0	37.8	31.2	38.8
WinoG	G	71.9	50.7	67.6	67.6	48.9	49.3	53.4	49.3	52.9	53.4	68.5
	L		51.4	67.8	67.0	52.7	66.6	61.0	50.0	69.1	61.0	67.0
CSQA	G	71.6	19.5	55.3	55.3	21.5	20.5	23.2	21.0	25.8	23.2	59.0
	L		20.2	59.1	57.7	19.2	52.4	30.5	18.4	57.7	30.5	60.3
MMLU	G	53.1	24.4	47.6	47.6	23.8	24.7	24.7	25.0	24.7	24.7	47.7
	L		24.1	48.6	46.9	24.4	42.6	34.8	00.4	48.0	34.8	47.6
GSM8K	G	58.3	00.2	32.0	32.0	00.0	00.0	00.2	00.0	00.0	00.2	32.1
	L		0.00	38.9	31.4	00.3	16.1	2.65	24.0	26.4	2.65	30.6

4.2 Hardware accelerator efficiency

We evaluate the hardware efficiency of our BSFU in terms of area (μm^2), energy efficiency (TOPS/W), and operating frequency. We synthesized the BSFU (Section 3.2) using Cadence tools and a 28 nm commercial PDK, and compare it at the compute-unit level against two recent designs targeting ternary-weight inference: BitROM [24] and BISDU [11]. To ensure a fair comparison, all area and power figures are scaled to a common technology node; BitROM’s values are reported for a 65 nm technology and thus scaled to 28 nm using $(28/65)^2$ node scaling. Table 3 summarizes the comparison.

BitROM is the closest peer: like TERINT-GEMV, it targets 1.58-bit ternary inference. Its Compute-in-ROM architecture keeps weights in on-chip SROM arrays and performs computation directly in memory, which avoids weight transfer but introduces local accumulators and multi-cycle read-compute sequences that add latency and area overhead. After scaling BitROM’s reported numbers from 65 nm to 28 nm, our design still achieves 19% higher energy efficiency (99.44 vs. 83.2 TOPS/W) with essentially the same area (13 952 vs. 13 905 μm^2). The efficiency advantage comes from our purely combinational datapath: the LUT resolves the 6-bit index in a single step with no accumulator state, and the downstream logic performs only additions and subtractions with no multiplier.

BISDU supports a range of bit widths and is designed to be reconfigurable for general low-precision arithmetic on microcontrollers, with ternary as one of several supported modes. That flexibility requires wider muxing, more complex control, and a datapath sized for the widest operand it may ever see. Even running at 1.15 GHz compared to our 100 MHz, BISDU reaches only 13.9 TOPS/W and occupies 21 856 μm^2 . TERINT-GEMV achieves 7.2 $\times$ better energy efficiency at 36% smaller area. The difference is that we never pay for configurability we do not need: the datapath is fixed to ternary-only operations from the ground up.

Table 3: Compute-unit-level efficiency comparison. BitROM area and power scaled from 65 nm to 28 nm (*).

	BSFU (ours)	BitROM	BISDU
Tech. Node	28 nm	65 nm *	28 nm
Frequency	100 MHz	100 MHz	1.15 GHz
Efficiency	99.44 TOPS/W	83.2 TOPS/W	13.9 TOPS/W
Area	13 952 μm^2	13 905 μm^2	21 856 μm^2

Table 4: Application-level performance comparison. Energy includes DRAM access at 3.7 pJ/bit (LPDDR5).

	TERINT-GEMV (SoC)	T-MAC (CPU)	LUT Tensor Core (GPU)
Throughput (Token/s)	32.47	15.62	133.87
Power (W)	1.74 W	10 W	150 W
Energy (J/tok)	0.065 J	0.66 J	1.12 J

4.3 End to End benchmarking

We compare against two baselines running 1.58-bit inference: T-MAC [22] (CPU-based LUT) and LUT Tensor Core (GPU-based). Our accelerator is modeled as a SoC component on the NVIDIA Jetson AGX Orin, with latency and energy obtained from a cycle-accurate simulator we developed (also open-sourced); energy accounts for accelerator switching power, CPU compute, and DRAM access at 3.7 pJ/bit (LPDDR5). Table 4 summarizes results.

Our TERINT-GEMV accelerator achieves 32.47 tokens/s at 1.74 W, a 2.1 $\times$ throughput improvement over T-MAC (15.62 tokens/s at 10 W) while consuming less than one-fifth of power. Against the GPU baseline, raw throughput is higher (133.87 tokens/s), but at 150 W, an 86 $\times$ power increase for a 4.1 $\times$ throughput gain. That tradeoff is economically indefensible for edge deployment.

At 0.065 J/tok, TERINT-GEMV is 10 $\times$ more efficient than T-MAC (0.66 J/tok) and 17 $\times$ more than the LUT Tensor Core (1.12 J/tok). These gains stem from multiplier-free compute, implicit zero-skipping, and a 25% reduction in weight memory from the 6-bit codebook, which directly reduces DRAM access overhead.

T-MAC and the GPU both pay for general-purpose hardware unnecessary for ternary operands: T-MAC uses multiply-accumulate pipelines, while the GPU leaves most CUDA cores idle during inference. Our BSFU-based TERINT-GEMV avoids both overheads through multiplier-free compute and implicit zero-skipping (§3.2.2).

5 Conclusion

We presented TERINT-GEMV, a hardware-software co-design closing the storage-waste and compute-mismatch gaps in 1.58-bit inference. A codebook pruning scheme compresses ternary weights to 1.5 bits/weight with VQ-aware fine-tuning, and a multiplier-free bit-serial accelerator ingests the 6-bit indices through an on-chip LUT with zero-skipping. Evaluated at 28 nm, it achieves 99.44 TOPS/W, a 90% energy-per-token reduction over T-MAC (0.065 vs. 0.66 J/token, including DRAM), and 2.1 $\times$ higher throughput at one-fifth the power. Addressing storage waste and arithmetic mismatch together thus unlocks gains neither software nor hardware achieves alone.

References

- [1] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432* (2013).
- [2] Vage Egiastian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme compression of large language models via additive quantization. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (ICML '24). JMLR.org, Article 488, 20 pages.
- [3] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Ariya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A Framework for Few-Shot Language Model Evaluation. doi:10.5281/zenodo.10256836
- [4] Mark Horowitz. 2014. 1.1 Computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 10–14. doi:10.1109/ISSCC.2014.6757323
- [5] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems* (Barcelona, Spain) (NIPS'16). Curran Associates Inc., Red Hook, NY, USA, 4114–4122.
- [6] Zijun Jiang and Yangdi Lyu. 2025. BNRV: A Lightweight SIMD Extension for Efficient BitNet Inference on RISC-V CPUs. In *2025 IEEE International Conference on Computer Design (ICCD)*. IEEE, 658–665.
- [7] Sangyeob Kim, Jungwan Lee, and Hoi-Jun Yoo. 2025. Slim-Llama: A 4.69mW Large-Language-Model Processor with Binary/Ternary Weights for Billion-Parameter Llama Model. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 421–423.
- [8] Bin Liu, Fengfu Li, Xiaoxing Wang, Bo Zhang, and Junchi Yan. 2023. Ternary Weight Networks. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 1–5. doi:10.1109/ICASSP49357.2023.10094626
- [9] Yun-Chen Lo and Ren-Shuo Liu. 2023. Bit-Serial Cache: Exploiting Input Bit Vector Repetition to Accelerate Bit-Serial Inference. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1109/DAC56929.2023.10247749
- [10] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. 2024. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764 [cs.CL] <https://arxiv.org/abs/2402.17764>
- [11] David Metz, Vineet Kumar, and Magnus Sjölander. 2023. BISDU: A Bit-Serial Dot-Product Unit for Microcontrollers. *ACM Trans. Embed. Comput. Syst.* 22, 5, Article 79 (Sept. 2023), 22 pages. doi:10.1145/3608447
- [12] Zhiwen Mo, Lei Wang, Jianyu Wei, Zhichen Zeng, Shijie Cao, Lingxiao Ma, Naifeng Jing, Ting Cao, Jilong Xue, Fan Yang, and Mao Yang. 2025. LUT Tensor Core: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 514–528. doi:10.1145/3695053.3731057
- [13] Dahoon Park, Hyungdong Park, Inguk Yeo, Jiyun Kim, Sunghyun Lee, Suhak Lee, Hyunseob Shin, Sung-il Pae, Deliang Fan, Jaeha Kung, and Kon-Woo Kwon. 2026. A Survey on Binary and Ternary Neural Networks and Their Realization in Compute-in-Memory for Edge Intelligence. *IEEE Internet of Things Journal* 13, 2 (2026), 3433–3458. doi:10.1109/JIOT.2025.3633487
- [14] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. 2024. Characterizing Power Management Opportunities for LLMs in the Cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 207–222. doi:10.1145/3620666.3651329
- [15] Georg Rutishauser, Joan Mihali, Moritz Scherer, and Luca Bonini. 2024. xTern: Energy-Efficient Ternary Neural Network Inference on RISC-V-Based Edge Systems. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. 206–213. doi:10.1109/ASAP61560.2024.00049
- [16] Moritz Scherer, Georg Rutishauser, Lukas Cavigelli, and Luca Benini. 2022. CUTIE: Beyond PetaOp/s/W Ternary DNN Inference Acceleration With Better-Than-Binary Energy Efficiency. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 4 (2022), 1020–1033. doi:10.1109/TCAD.2021.3075420
- [17] Haoxuan Shan, Cong Guo, Chiyue Wei, Feng Cheng, Junyao Zhang, Hai Li, and Yiran Chen. 2025. Platinum: Path-Adaptable LUT-Based Accelerator Tailored for Low-Bit Weight Matrix Multiplication. *arXiv preprint arXiv:2511.21910* (2025).
- [18] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Vikas Chandra, and Hadi Esmaeilzadeh. 2018. Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Network. In *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1–6, 2018*. IEEE, Los Angeles, CA, USA, 764–775.
- [19] Tejas Vaidhya, Ayush Kaushal, Vineet Jain, Francis Couture-Harpin, Prashant Shishodia, Majid Behbahani, Irina Rish, and Yuriy Nevmyvaka. 2025. Scaling Laws and Efficient Inference for Ternary Language Models. In *Sparsity in LLMs (SLLM): Deep Dive into Mixture of Experts, Quantization, Hardware, and Inference*. <https://openreview.net/forum?id=h28ryFZV0u>
- [20] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2017. Neural discrete representation learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6309–6318.
- [21] Jinheng Wang, Hansong Zhou, Ting Song, Shijie Cao, Yan Xia, Ting Cao, Jianyu Wei, Shuming Ma, Hongyu Wang, and Furu Wei. 2025. Bitnet.cpp: Efficient Edge Inference for Ternary LLMs. arXiv:2502.11880 [cs.LG] <https://arxiv.org/abs/2502.11880>
- [22] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (EuroSys '25). Association for Computing Machinery, New York, NY, USA, 278–292. doi:10.1145/3689031.3696099
- [23] Yuzhuang Xu, Xu Han, Zonghan Yang, et al. 2024. OneBit: Towards Extremely Low-Bit Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc. doi:10.1145/079017-2122
- [24] Wenlun Zhang, Xinyu Li, Shimpei Ando, and Kentaro Yoshioka. 2025. BitROM: Weight Reload-Free CiROM Architecture Towards Billion-Parameter 1.58-bit LLM Inference. arXiv:2509.08542 [cs.AR] <https://arxiv.org/abs/2509.08542>